Week 9 Level 1: Introduction

Variables, Input and Choices

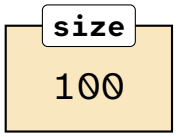
Programs can ask questions, remember the answers and decide what to do.

What you will learn

- How a **variable** remembers a value.
- How to ask a question with `input`, and turn the answer into a number with `int`.
- How `if`, `elif` and `else` let a program choose.
- Which questions a program should never ask.

The big idea: labeled boxes

Remember the `score` variable in Scratch? A variable is a box with a label. You put something in it, and you can use it later.

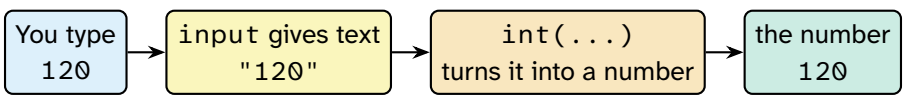


`size = 100` puts `100` in the box labeled `size`.
`t.forward(size)` takes it out and uses it: `100` steps.

One equals sign = means **put in the box**. (Two equals signs == mean something else — see below!)

Asking a question

`input` asks a question in the Shell and waits. Click in the Shell, type your answer and press `Return`. But watch out: `input` **always** gives back **text**, even when you type a number.



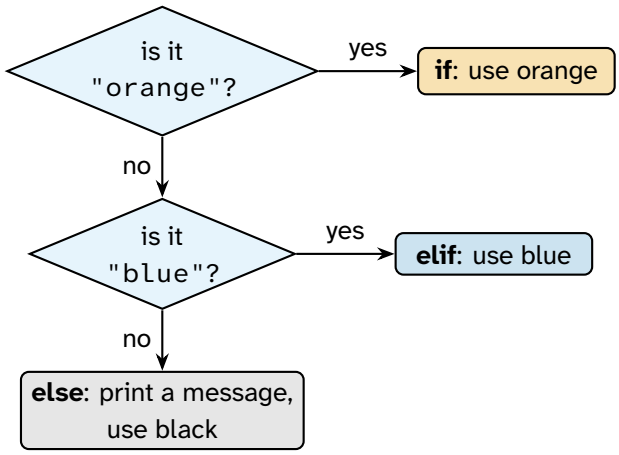

```
answer = input("How big should the square be? ")
size = int(answer)
```

Exercise 9.1: Square of Any Size

Folder: `week09/ex-size-it`. Run the starter and type `120` in the Shell. A red `TypeError`! The turtle cannot walk the text `"120"`. Fix it with `size = int(answer)`. Try 50, 150 and 250.

Making choices: if, elif, else

A program can choose what to do. Python checks the rules from the top. The **first** rule that is true wins, and the others are skipped. `else` catches everything that is left.




```
if color_name == "orange":
    color = "#E69F00"
elif color_name == "blue":
    color = "#0072B2"
else:
    print("I do not know that color. I will use black.")
    color = "#000000"
```

Every `if`, `elif` and `else` line ends with a colon, and the lines under it are pushed in 4 spaces — just like loops.

Comparing

Sign	Means	Example that is true
`==`	is the same as	`"blue" == "blue"`
`!=`	is not the same as	`"Blue" != "blue"` (Python is picky!)
`<`	is less than	`3 < 10`
`>`	is more than	`25 > 10`

Exercise 9.2: Pick a Color

Folder: `week09/ex-color-choice`. The program knows orange and blue. Add an `elif` for `"green"` (`"#009E73"`) and an `else` that prints a friendly message and uses black. Test with `orange`, `blue`, `green` and `pizza`. The color name is written under the circle, so you can tell the colors apart even without seeing color.

Exercise 9.3: How Many Stars?

Folder: `week09/ex-star-row`. Ask how many stars (1 to 10). Before drawing: **if** the number is more than 10, print a message and use 10; **elif** it is less than 1, print a message and use 1; **else** print `"Coming right up!"`. Test with 3, 25 and 0.

Questions a program should NOT ask

Check one box for each question.

Question	OK	Never
What is your favorite color?	☐	☐
Where do you live?	☐	☐
How many stars should I draw?	☐	☐
What is your password?	☐	☐
What is your hero name?	☐	☐
What school do you go to?	☐	☐
What is your full name?	☐	☐
What is your favorite animal?	☐	☐

Our rule: ask about **choices** (colors, sizes, numbers), never about **who you are**. If an app or game asks for personal details, stop and ask a grown-up.

Vocabulary box

Variable A labeled box that remembers a value, like `size = 100`.

input Asks a question in the Shell and gives back the answer as text.

int Turns text like `"120"` into the number 120.

Text (string) Letters or numbers inside quote marks, like `"blue"` or `"120"`.

if / elif / else Python's way to choose: "if this, else if that, otherwise..."

Comparison A question with a yes/no answer, like `stars > 10`.

Reflection

Think of a game you know. Where does it ask you something and then decide what happens?

Why does `input` need `int` for sizes, but not for color names?

Week 9

Mini Math Quiz

Level 1: Introduction

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Mini Math Quiz (about 10 minutes)

Open `week09/hw-math-quiz/starter.py`. No turtle this time: everything happens in the Shell.

1. Press Run and answer question 1 in the Shell. Read how it works: `input`, then `int`, then `if/else`, and `score = score + 1` when the answer is right.
2. Add question 2 the same way: "What is 9 + 15?" (the answer is 24).
3. At the end, print the score: `print("You scored", score, "out of 2")`.
4. Test it twice: once with right answers, once with wrong ones. Then run `check_my_work.py`.

No computer at home? Write question 2 here (copy the pattern of question 1):

★★ Challenge (want more?): Perfect score

Add a third question of your own. Change "out of 2" to "out of 3". Then, at the very end, add an `if` that prints a special message when `score == 3`. Remember: two equals signs ask a question!

Creative (make it yours): Choose your path

Write a tiny adventure: the program asks "You find a door and a tunnel. Which do you choose?" and uses `if/elif/else` to print a different ending for `door`, `tunnel` and anything else. Plan it here first:

Remember the rule: our programs ask about **choices**, never about who you are (no names, addresses, schools or passwords).

Self-check: tick each box when it is true

- ☐ My quiz asks 2 questions and says `Correct!` for right answers.
- ☐ The last line shows my score.
- ☐ I know why we need `int(...)` around `input(...)` for numbers.
- ☐ I know the difference between = (put in the box) and == (are they the same?).